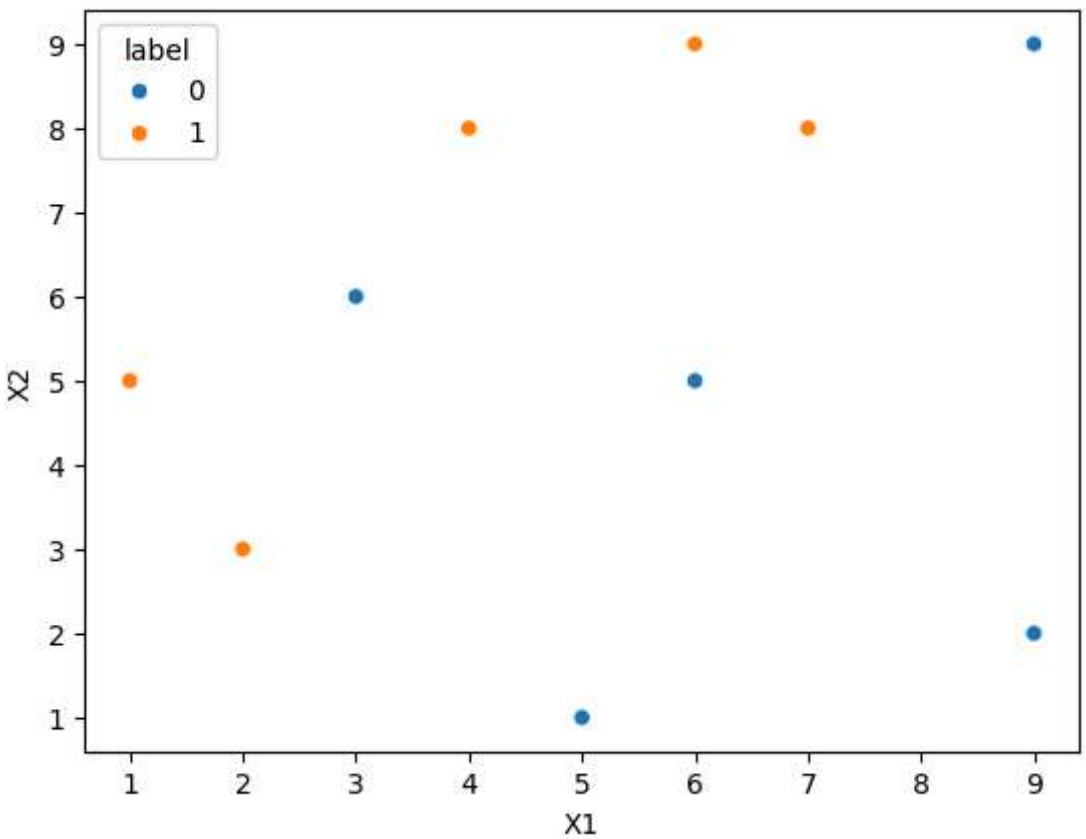
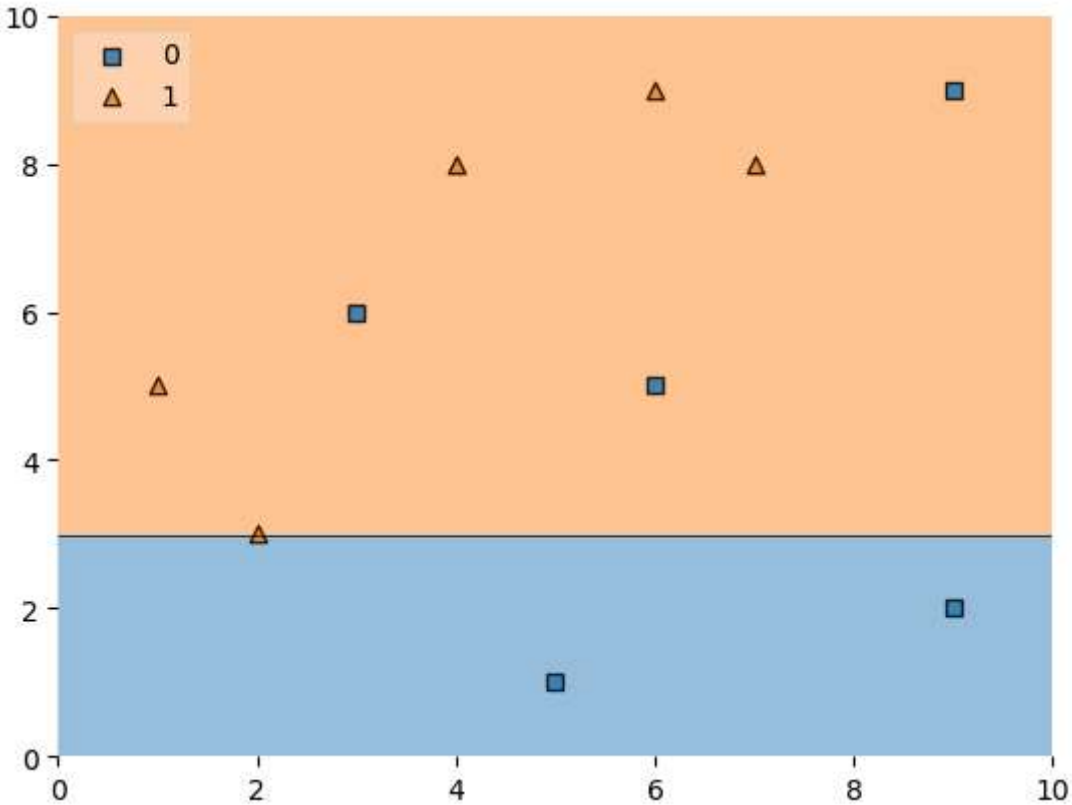
In [3]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree
from mlxtend.plotting import plot_decision_regions

# Create dataset
df = pd.DataFrame()

df['X1'] = [1,2,3,4,5,6,6,7,9,9]
df['X2'] = [5,3,6,8,1,9,5,8,9,2]
df['label'] = [1,1,0,1,0,1,0,1,0,0]

# Scatter plot
sns.scatterplot(x=df['X1'], y=df['X2'], hue=df['label'])
plt.show()
```



In [4]:
```
df['weights'] = 1/df.shape[0]

# Prepare data
x = df.iloc[:,0:2].values
y = df.iloc[:,2].values

# First Decision Tree
dt1 = DecisionTreeClassifier(max_depth=1)
dt1.fit(x,y)
```

```
# Decision boundary
plot_decision_regions(x, y, clf=dt1, legend=2)
plt.show()
```

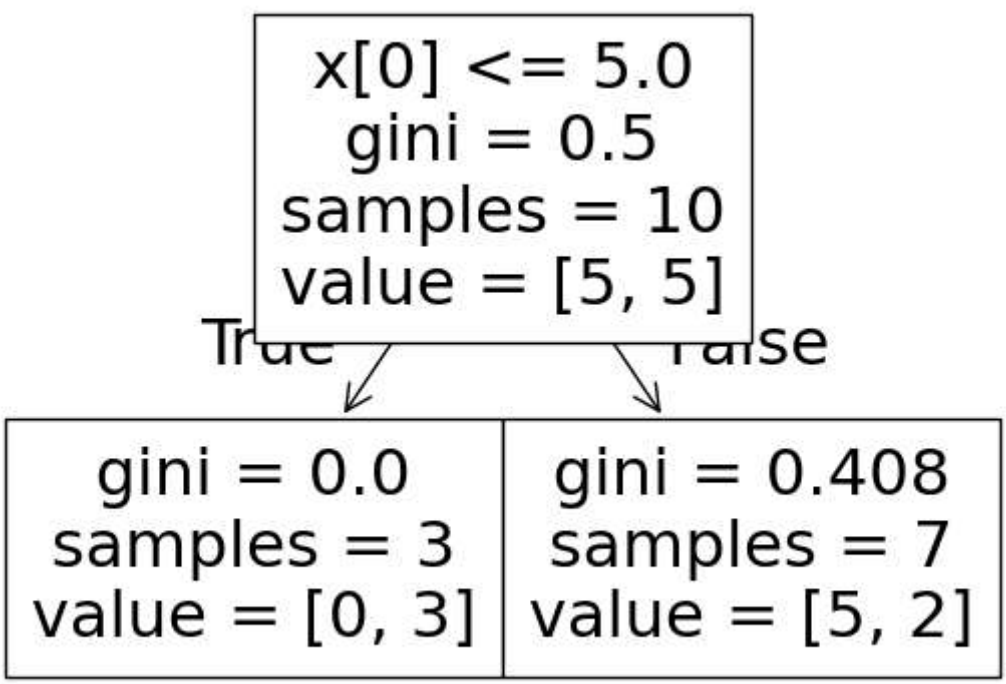


In [5]:
```
# Predictions
df['y_pred'] = dt1.predict(x)

# Function to calculate model weight
def calculate_model_weight(error):
    return 0.5*np.log((1-error)/(error))

# Model weight
alpha1 = calculate_model_weight(0.3)
print(alpha1)

# Update row weights
def update_row_weights(row, alpha=0.423):
    if row['label'] == row['y_pred']:
        return row['weights'] * np.exp(-alpha)
    else:
        return row['weights'] * np.exp(alpha)

df['updated_weights'] = df.apply(update_row_weights, axis=1)

# Normalize weights
df['normalized_weights'] = df['updated_weights']/df['updated_weights'].sum()

# Cumulative sum
```

```
df['cumsum_upper'] = np.cumsum(df['normalized_weights'])
df['cumsum_lower'] = df['cumsum_upper'] - df['normalized_weights']
```

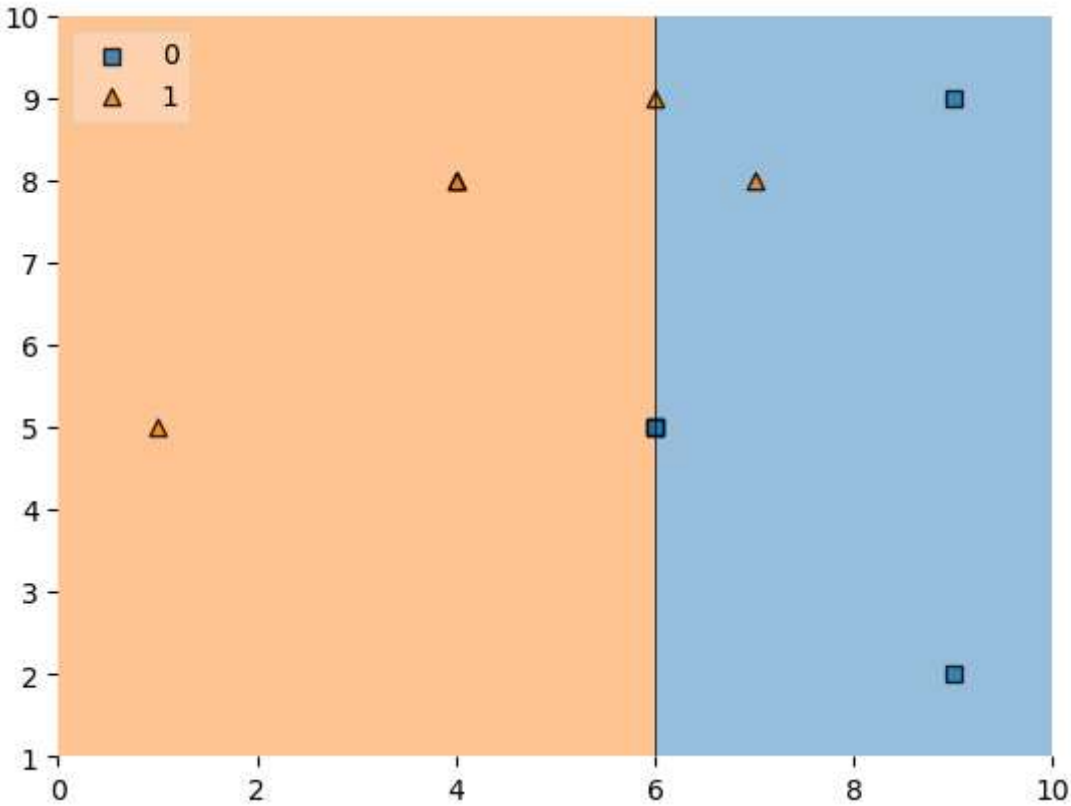
```
0.42364893019360184
```

In [7]:
```
# Function to create new dataset
def create_new_dataset(df):
    indices = []
    for i in range(df.shape[0]):
        a = np.random.random()
        for index, row in df.iterrows():
            if row['cumsum_upper'] > a and a > row['cumsum_lower']:
                indices.append(index)
    return indices

# Sampling
index_values = create_new_dataset(df)

second_df = df.iloc[index_values,[0,1,2,3]]

# Second Decision Tree
dt2 = DecisionTreeClassifier(max_depth=1)

x = second_df.iloc[:,0:2].values
y = second_df.iloc[:,2].values

dt2.fit(x,y)

plot_tree(dt2)
plt.show()
```



In [8]:
```
plot_decision_regions(x, y, clf=dt2, legend=2)
```

```
plt.show()
```



In [9]:
```
# Predictions
second_df['y_pred'] = dt2.predict(x)

# Model weight
alpha2 = calculate_model_weight(0.1)
print(alpha2)

# Update weights
def update_row_weights2(row, alpha=1.09):
    if row['label'] == row['y_pred']:
        return row['weights'] * np.exp(-alpha)
    else:
        return row['weights'] * np.exp(alpha)

second_df['updated_weights'] = second_df.apply(update_row_weights2, axis=1)

second_df['normalized_weights'] = second_df['updated_weights']/second_df['updated_w

second_df['cumsum_upper'] = np.cumsum(second_df['normalized_weights'])
second_df['cumsum_lower'] = second_df['cumsum_upper'] - second_df['normalized_weigh

# Third model weight example
alpha3 = calculate_model_weight(0.7)

print(alpha1, alpha2, alpha3)
```

```
1.0986122886681098
0.42364893019360184 1.0986122886681098 -0.4236489301936017
```

In [10]:
```
# Test queries
query = np.array([1,5]).reshape(1,2)

print(dt1.predict(query))
print(dt2.predict(query))

query = np.array([9,9]).reshape(1,2)

print(dt1.predict(query))
print(dt2.predict(query))
```

```
[1]
[1]
[1]
[0]
```

In [ ]: